À la recherche de mon député idéal !!

CMI

Période :

30 Septembre 2024 – 31 Décembre 2024

Lien GitHub :

magentahttps://github.com/zhrssd/projet-info.git

Membres du projet :

Fatima SAADNA - 43004972

Imene BEN EL KHAZNADJI - 43001905

Contents

red1 Introduction : 3
red1.1 Objectif du projet 3

red2 Environnement de travail : 3

red3 Description du projet et objectifs : 3
red3.1 Collecte et préparation des données 3
red3.2 Analyse des données collectées 3
red3.3 Affichage des résultats 4

red4 Bibliothèques, outils et technologies : 4

red5 Travail réalisé : 5
red5.1 Organisation des fichiers 5
red5.2 Importation des données 6
red5.3 Transformation et structuration des données 6
red5.4 Analyse et clustering 7
red5.5 Résultats du clustering : le dendrogramme 7
red5.6 Interface utilisateur 8
red5.7 Interface utilisateur 9
red5.8 Analyse de `app.py` 9
red5.9 Analyse de `index.html` 10

red6 Difficultés rencontrées : 10

red7 Bilan : 10
red7.1 Conclusion 10

red8 Annexes 12
red8.1 Présentation de l'interface utilisateur 12

1 Introduction :

1.1 Objectif du projet

Tout d'abord, nous tenons à remercier notre professeur pour cette idée de projet que nous avons adorée. Ce sujet nous a réellement motivés et permis d'apprendre tout en travaillant sur un thème aussi original qu'enrichissant.

Dans un contexte où la politique semble souvent influencée par des facteurs extérieurs tel que l'influence plutôt que par des convictions profondes, il est crucial de vérifier si les élus défendent réellement les idées qu'on leur attribue. Beaucoup d'électeurs suivent un courant ou un député par habitude ou ouï-dire, sans savoir si leurs convictions sont alignées. Notre projet vise à lever cette confusion.

Concrétement le programme extrait les votes des députés depuis des fichiers XML et crée un tableau associant chaque député à ses positions (pour, contre, abstention) sur chaque loi. Ces données sont transformées en chiffres pour analyser les similarités grâce à un algorithme de clustering hiérarchique.

Ce travail se distingue par son interface utilisateur : un QCM créé avec Flask où l'utilisateur répond à des questions sur des lois. Le programme compare ensuite ses réponses à celles des députés et détermine celui qui partage le plus ses convictions, sans influence médiatique.
Nous allons maintenant vous détailler chaque partie de notre travail. Pour commencer, voici comment nous avons collaborer sur notre projet.

2 Environnement de travail :

Au début, nous avons divisé le travail, avec Fatima s'occupant des données et du site, et Imène du code. Nous échangions principalement par mail et téléphone, mais avons constaté que nos rencontres en bibliothèque étaient plus productives, facilitant la résolution rapide des problèmes. Le découpage initial s'est avéré difficile, car nos tâches se complétaient, et nous avons donc décidé de travailler ensemble sur chaque étape du projet pour avancer efficacement. Travailler en équipe a facilité la résolution des obstacles rencontrés.

3 Description du projet et objectifs :

3.1 Collecte et préparation des données

Nous avons collecté les données de la 15 législature depuis les sites officiels de l'Assemblée nationale : magentaListe simplifiée des mandats et magentaS-

crutins, au format XML. Les fichiers ont été centralisés, triés et structurés pour organiser les informations sur les députés et leurs votes, préparant ainsi les données pour les analyses.

3.2 Analyse des données collectées

Dans la deuxième phase, nous avons analysé les données collectées. Les votes ont été convertis en valeurs numériques ("pour" = 1, "contre" = -1, "abstention" = 0) pour un traitement statistique. À l'aide de la bibliothèque scipy, un clustering hiérarchique a regroupé les députés selon la similarité de leurs votes. Seuls les groupes de 20 membres ou plus ont été conservés et visualisés sous forme de dendrogramme.

3.3 Affichage des résultats

Enfin, nous avons conçu une page web interactive nous avons utilisé Flask pour créer une page web interactive, permettant de gérer les calculs côté serveur. L'utilisateur y simule ses votes via un QCM. Le système identifie les 3 députés ayant les votes les plus proches et affiche leurs informations (nom, région, parti).

4 Bibliothèques, outils et technologies :

Pour réaliser ce projet, nous avons utilisé différents outils et bibliothèques qui nous ont permis de collecter, traiter, analyser et afficher les données. Voici les technologies les plus importantes utilisées :

Backend :

Python : Langage principal pour le serveur avec Flask et les utilitaires d'analyse de données.
Flask : Gère le serveur web.
NumPy, Pandas : Manipulation et analyse des données.
Matplotlib : Visualisations, notamment les dendrogrammes.
Scipy : Clustering hiérarchique pour regrouper les députés.
Scikit-learn : Calcul de la similarité cosinus.
Logging et XML : Traitement des fichiers XML et gestion des erreurs.

Frontend :

HTML : Structure de la page web.
CSS : Personnalisation de style et animations (dans <style>).
Bootstrap CSS (via CDN) : Design moderne et responsive.
JavaScript : Interactions dynamiques sur la page.
Bibliothèques utilisées :
jQuery : Manipulation du DOM et requêtes AJAX.
Bootstrap JS : Fonctionnalités interactives (tooltips, navbar, etc.).
Toastr : Affichage de notifications.
Chart.js : Création de graphiques interactifs.

Jinja2 : Moteur de templates intégré à Flask pour rendre les données dynamiques dans le HTML.
Données :

XML : Format des fichiers de données (députés et votes) analysés dans le backend.
Exportation :
CSV : Format généré pour permettre à l'utilisateur de télécharger les résultats.

ChatGPT-01 a été un précieux assistant tout au long du projet, nous aidant à résoudre des problèmes techniques, à créer du code, et à maîtriser des concepts clés, notamment pour l'analyse des données, la création de l'interface web et la mise en œuvre du clustering.

5 Travail réalisé :

Notre projet est organisé en plusieurs fichiers, permettant une meilleure modularité avec Flask.

Nous avons importé les bibliothèques nécessaires, dont les plus importantes ont été détaillées précédemment. Ensuite, nous avons traité 1 242 fichiers pour les députés (noms et identifiants uniques) et 4 417 fichiers pour les votes (positions sur différentes lois : pour, contre, abstention).

En combinant ces données, nous avons pu associer chaque député à ses votes et identifier précisément quelles lois ont été votées et par qui.

```
2025-01-03 19:37:35,314 - INFO - Nombre de fichiers XML (dép
 dans C:\Users\Admin\Desktop\projet l2\AMO20_dep_sen_min_tou
dats_et_organes_XV.xml\xml\acteur : 1242
2025-01-03 19:37:36,834 - INFO - Nombre de fichiers XML (vot
ans C:\Users\Admin\Desktop\projet l2\xml : 4417
```

5.1 Organisation des fichiers

L'organisation des fichiers dans le projet est structurée comme suit :

```
projet
 _requirements.txt
 _data
    __ acteur (1242 fichiers XML - députés)
   __ xml (4417 fichiers XML - votes)
 _static
```

```
   __ dendrogrammes (images générées)
_ templates
    __index.html (Interface principale)
    __about.html (Page à propos)
    __404.html (Page d'erreur 404)
    __ 500.html (Page d'erreur 500)
_ app.py (Application Flask)
_ utils.py (Fonctions d'analyse)
```

Chaque dossier et fichier a un rôle précis :

- **data/acteur** : Contient les données XML des députés (1242 fichiers).
- **data/xml** : Regroupe les données XML des votes (4417 fichiers).
- **static** : Stocke les fichiers statiques, comme les dendrogrammes générés.
- **templates** : Contient les fichiers HTML pour l'interface utilisateur, notamment les pages principales et les pages d'erreur 404 et 500.
- **app.py** : Le fichier principal qui exécute l'application Flask.
- **utils.py** : Regroupe les fonctions pour analyser les données et effectuer les traitements nécessaires.

5.2 Importation des données

Les données ont été extraites et organisées grâce à deux fonctions principales :

- `parse_deputes` : Extrait les informations des députés (nom, prénom, identifiant unique) à partir des fichiers XML dans `data/acteur`.

```python
def parse_deputes(dossier_xml_deputes: str, namespace: dict) -> dict:
    deputes_dict = {}
    try:
        xml_files = [f for f in os.listdir(dossier_xml_deputes) if f.endswith('.xml')]
        logger.info(f"Nombre de fichiers XML (députés) dans {dossier_xml_deputes} : {len(xml_files)}")

        for fichier in xml_files:
            chemin_fichier = os.path.join(dossier_xml_deputes, fichier)
            try:
                tree = ET.parse(chemin_fichier)
                root = tree.getroot()

                uid = root.find('.//ns:uid', namespace)
                nom = root.find('.//ns:nom', namespace)
                prenom = root.find('.//ns:prenom', namespace)

                if uid is not None and nom is not None and prenom is not None:
                    name = f"{prenom.text} {nom.text}"
                    deputes_dict[uid.text] = {
                        "name": name,
                        "uid": uid.text
                    }
                else:
                    logger.warning(f"Informations manquantes dans {chemin_fichier}")
```

- `parse_votes` : Extrait les votes des députés (*pour*, *contre*, *abstention*) pour chaque scrutin dans `data/xml` et organise ces données dans un tableau.

Ces données ont ensuite été combinées pour associer chaque député à ses votes, préparant ainsi l'analyse.

5.3 Transformation et structuration des données

Les données extraites ont été transformées et structurées pour faciliter leur analyse :

- Les votes (*pour*, *contre*, *abstention*) ont été remplacés par des valeurs numériques :
  - *Pour* : $+1$
  - *Contre* : -1
  - *Abstention* : 0

```python
# Mapping des votes en valeurs numériques
VOTE_MAPPING = {
    'pour': 1,
    'contre': -1,
    'abstention': 0,
    'non_votant_volontaire': np.nan,
    'absent': np.nan
}
```

- Les données combinées des députés et des votes ont été organisées dans un tableau, avec :
  - Les colonnes représentant les lois.
  - Les lignes correspondant aux députés.
  - Les cellules indiquant la position de chaque député (sous forme numérique).

5.4 Analyse et clustering

Pour regrouper les députés ayant des comportements de vote similaires, un clustering hiérarchique a été réalisé. Ce processus est basé sur la méthode de Ward, qui minimise la variance intra-groupe.

- **Clustering hiérarchique** : Les votes des députés ont été utilisés pour générer des groupes en fonction de leur proximité.
- **Dendrogramme** : Une visualisation a été produite pour représenter les regroupements des députés.

- **Filtrage des groupes** : Seuls les groupes contenant au moins 20 députés ont été retenus.

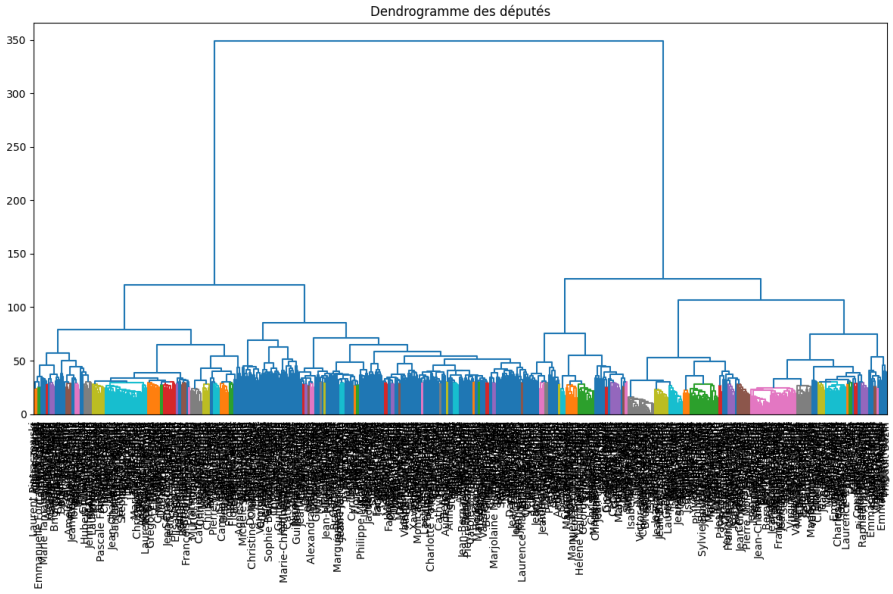
```python
def cluster_deputes(votes_df_numeric_filtered: pd.DataFrame,
                    deputes_dict: dict,
                    cluster_threshold: int,
                    output_filename: str) -> dict:
    try:
        Z = linkage(votes_df_numeric_filtered.fillna(0), method='ward')
        clusters = fcluster(Z, cluster_threshold, criterion='maxclust')

        grouped_deputes = {}
        for idx, cluster_id in enumerate(clusters):
            uid = votes_df_numeric_filtered.index[idx]
            deputy_info = deputes_dict.get(uid, {})
            deputy_name = deputy_info.get("name", uid)
            grouped_deputes.setdefault(f"Groupe {cluster_id}", []).append(deputy_name)

        dendrogram_dir = os.path.join(os.getcwd(), 'static', 'dendrogrammes')
        os.makedirs(dendrogram_dir, exist_ok=True)

        plt.figure(figsize=(12, 8))
        dendrogram(
```

5.5 Résultats du clustering : le dendrogramme

Le dendrogramme ci-dessous représente les résultats du clustering hiérarchique appliqué aux votes des députés. Voici les points essentiels à comprendre :

- L'axe horizontal représente les députés (avec leurs noms).
- L'axe vertical montre la distance entre les clusters (groupes).
- Les députés sont regroupés en fonction de la similarité de leurs votes.
- Chaque couleur correspond à un cluster contenant au moins 20 députés (critère minimum défini dans le code).
- Plus une branche monte haut avant de se diviser, plus les groupes ou individus qu'elle relie sont différents en termes de votes. Les branches courtes signifient que les groupes sont très similaires.
- Un seuil de dissimilarité a été défini (ici 40 dans le code). Les groupes formés respectent ce seuil, garantissant que les membres d'un cluster ont des votes similaires.

Ce graphique permet de visualiser les comportements politiques similaires et d'identifier les grandes tendances au sein de l'Assemblée Nationale durant la 15 législature.



Après avoir extrait, convertit, analyser et trier les données, nous passons désormais à la création du site HTML qui constitue la majorité de notre code. Ce site inclut le qcm qui permet à l'utilisateur de "voter" aux lois et ainsi découvrir quels député partagent le plus de similitudes avec ses choix.

5.6 Interface utilisateur

Une interface web interactive a été développée pour rendre les résultats de l'analyse accessibles :

- **Formulaire QCM** : Permet à l'utilisateur de simuler des votes sur différentes lois.

- **Résultats dynamiques** : Affiche les 3 députés les plus similaires à l'utilisateur, avec des informations comme leur nom, leur région et leur parti.

```python
def compute_top_3_similar_deputes(user_votes: dict,
                                  votes_df: pd.DataFrame,
                                  deputes_dict: dict) -> list:
    if not user_votes:
        logger.warning("Aucune réponse fournie par l'utilisateur.")
        return []

    laws_answered = list(user_votes.keys())
    user_vector = [
        VOTE_MAPPING.get(choice, 0) for choice in user_votes.values()
    ]
    user_vector = np.array(user_vector).reshape(1, -1)

    try:
        sub_df = votes_df[laws_answered].replace(VOTE_MAPPING)
    except KeyError as e:
        logger.error(f"Loi non trouvée dans les données de votes : {e}")
        return []

    sub_df = sub_df.fillna(0)

    if sub_df.empty:
        logger.warning("DataFrame filtré est vide après remplacement des valeurs.")
        return []

    similarity_scores = cosine_similarity(user_vector, sub_df.values)[0]
```

- **Graphique de similarité** : Visualise les scores des députés via un graphique généré par `Chart.js`.
- **Téléchargement des résultats** :
  - Résultats en CSV.
  - Graphiques en PNG.
- **Erreurs personnalisées** : Les pages `404.html` et `500.html` gèrent les erreurs de manière conviviale.

5.7 Interface utilisateur

Une interface web interactive a été développée pour rendre les résultats de l'analyse accessibles :

- **Formulaire QCM** : Permet à l'utilisateur de simuler des votes sur différentes lois.
- **Résultats dynamiques** : Affiche les 3 députés les plus similaires à l'utilisateur, avec des informations comme leur nom.
- **Graphique de similarité** : Visualise les scores des députés via un graphique généré par `Chart.js`.
- **Téléchargement des résultats** :
  - Résultats en CSV.
  - Graphiques en PNG.

Répondez aux Lois (QCM)

la declaration de politique generale du Gouvernement de M. Edouard Philippe (application de l'article 49, alinea premier, de la Constitution). Pour Contre Abstention

l'amendement n° 11 de M. Bernalicis à l'article unique de la proposition de loi renforçant la lutte contre les rodéos motorisés (première lecture). Pour Contre Abstention

l'amendement n° 2 de M. Pauget à l'article unique de la proposition de loi renforçant la lutte contre les rodéos motorisés (première lecture). Pour Contre Abstention

l'article unique de la proposition de loi renforçant la lutte contre les rodéos motorisés (première lecture). Pour Contre Abstention

l'ensemble de la proposition de loi renforçant la lutte contre les rodéos motorisés (première lecture). Pour Contre Abstention

la motion de rejet préalable, déposée par M. André Chassaigne, du projet de loi autorisant la ratification de l'accord de partenariat sur les relations et la coopération entre l'Union européenne et ses États membres d'une part, et la Nouvelle-Zélande, d'autre part (première lecture).
Pour Contre Abstention

l'article unique du projet de loi autorisant la ratification de l'accord de partenariat sur les relations et la coopération entre l'Union européenne et ses États membres d'une part, et la Nouvelle-Zélande, d'autre part (première lecture).
Pour Contre Abstention

l'ensemble du projet de loi pour la confiance dans la vie publique (première lecture). Pour Contre Abstention

l'amendement n° 2199 de Mme Obono avant l'article premier du projet de loi constitutionnelle pour une démocratie plus représentative, responsable et efficace (première lecture).
Pour Contre Abstention

l'amendement n° 199 de M. Lagarde avant l'article premier du projet de loi constitutionnelle pour une démocratie plus représentative, responsable et efficace (première lecture).
Pour Contre Abstention

Code Html qui affiche le QCM.

5.8 Analyse de `app.py`

Le fichier `app.py` constitue le cœur de l'application Flask, gérant les routes et les interactions entre le backend et le frontend.

- **Importation des bibliothèques** : Flask pour l'application web, ainsi que Pandas, Matplotlib et NumPy pour le traitement des données.
- **Chargement des données** : La fonction `load_data` charge et organise les données des députés et des votes dès le démarrage de l'application.
- **Routes principales** :
  - `/` : Affiche la page principale avec le formulaire QCM.
  - `/submit` : Traite les votes simulés et renvoie les députés les plus similaires.
  - `/dendrogrammes/<filename>` : Permet d'afficher les dendrogrammes dans l'interface.
- **Exécution** : L'application est lancée sur le port 8000.

5.9 Analyse de `index.html`

Le fichier `index.html` constitue le template principal de l'interface utilisateur. Il propose plusieurs fonctionnalités interactives :

- **Barre de navigation** : Permet de naviguer entre la page d'accueil et la page "À propos".

- **Formulaire QCM** : Affiche les lois pour lesquelles l'utilisateur peut simuler des votes (*pour*, *contre*, *abstention*).

- **Résultats dynamiques** : Affiche les 3 députés les plus similaires dans un tableau.

- **Graphique de similarité** : Généré avec `Chart.js`, il visualise les scores des députés.

- **Téléchargements** :

  – Résultats disponibles en CSV.

  – Graphique téléchargeable au format PNG.

6 Difficultés rencontrées :

Plusieurs difficultés ont été rencontrées lors du développement de ce projet :

- **Simplification du code** : Initialement, tout le projet était contenu dans un seul fichier Python de plus de 1000 lignes, ce qui rendait la gestion complexe. Pour améliorer la lisibilité et la maintenance, nous avons réorganisé le code en plusieurs fichiers distincts (modularisation).

- **Volume important de données** : L'un des premiers obstacles a été l'importante quantité de fichiers XML à traiter. Chaque fichier contient de nombreuses informations sur les députés et leurs votes sur diverses lois. Cela a nécessité une organisation rigoureuse pour éviter d'utiliser les mauvaises données ou de mal associer un vote à un élu. Une petite erreur dans l'identification ou un champ manquant aurait pu compromettre la fiabilité des résultats.

- **Clustering hiérarchique** : Construire des groupes de députés en fonction de la similarité de leurs votes s'est révélé gourmand en ressources. Avec un grand nombre de députés et de lois, les calculs de distance entre chaque paire de députés augmentent rapidement, ce qui allonge le temps de traitement. Pour gérer cette contrainte, nous avons optimisé le remplissage et le nettoyage des données, tout en réduisant parfois le nombre de lois considérées pour conserver un temps de calcul raisonnable sans trop perdre en précision.

7 Bilan :

7.1 Conclusion

En conclusion, ce projet a représenté un véritable défi, principalement en raison du volume important de données à traiter et de la complexité des concepts impliqués. L'une des principales difficultés a été de regrouper les députés en fonction de leurs votes, car les notions de clustering hiérarchique et de matrices de corrélation étaient nouvelles pour nous. Ces étapes ont demandé une grande réflexion et de nombreux ajustements techniques. Cependant, malgré ces obstacles, ce projet nous a permis de développer des quelques connaissances en traitement de données et en algorithmique, tout en nous offrant une expérience pratique. Le résultat final, un site capable de déterminer quel député correspond le mieux aux convictions d'un utilisateur, a été une source de satisfaction.

8 Annexes

Nous allons maintenant vous expliquer le fonctionnement de notre site. Pour commencer, lorsqu'on lance notre code, le lien d'un serveur de développement s'affiche dans notre terminal

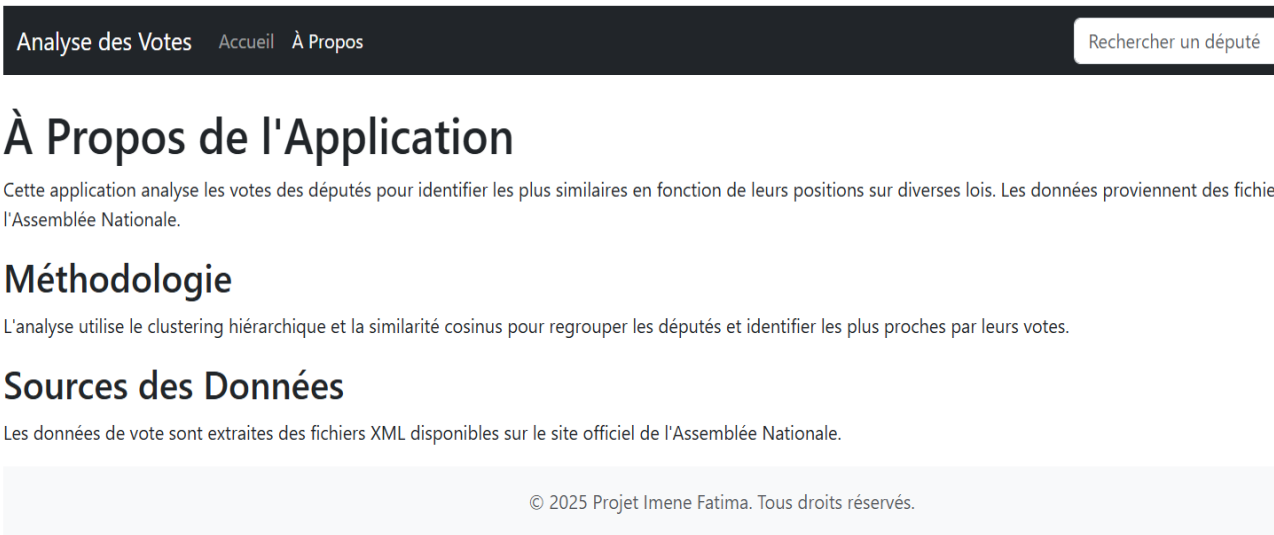
```
 votes_df_numeric = VOTES_DF.replace(VOTE_MAPPING)
2025-01-04 23:47:23,091 - ERROR - Nombre de groupes après filtrage (≥ 20 membres) : 16
2025-01-04 23:47:23,091 - ERROR - Données chargées avec succès.
 * Serving Flask app 'app'
 * Debug mode: off
2025-01-04 23:47:23,103 - INFO - WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI serve
 * Running on http://127.0.0.1:8000
2025-01-04 23:47:23,103 - INFO - Press CTRL+C to quit
```

Voici l'entête de notre site :

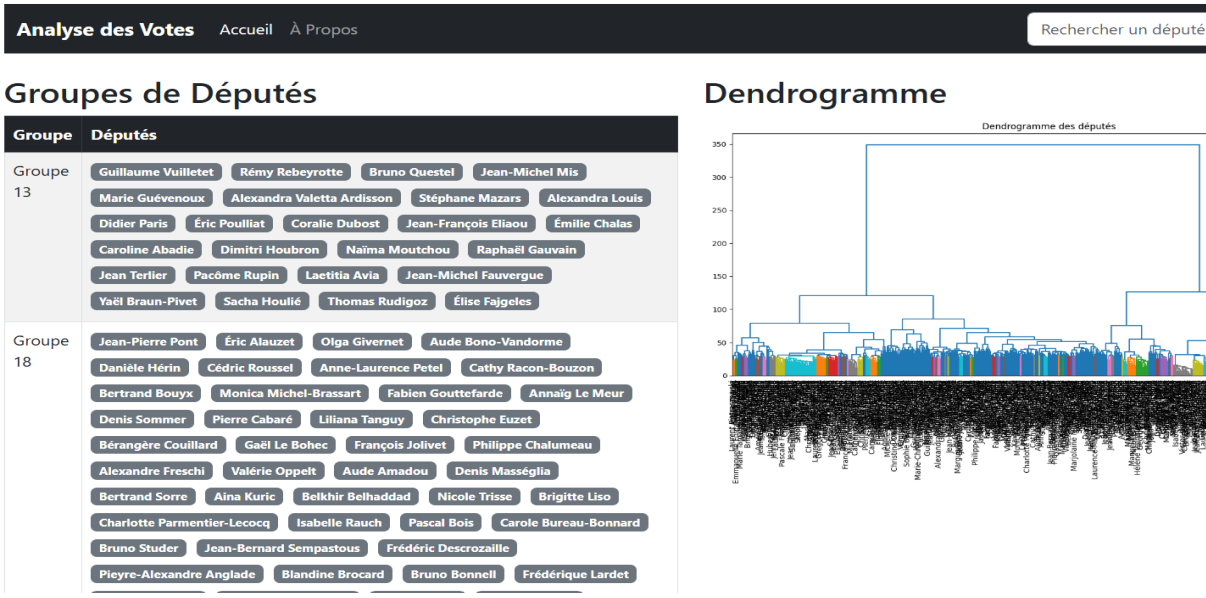
8.1 Présentation de l'interface utilisateur

Voici une présentation de notre site avec des captures d'écran illustrant ses différentes sections et fonctionnalités :

- **En-tête du site** :

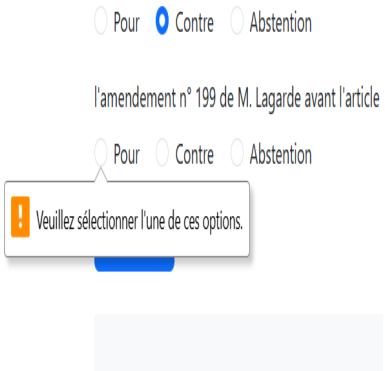


- **Présentation générale de la page** :

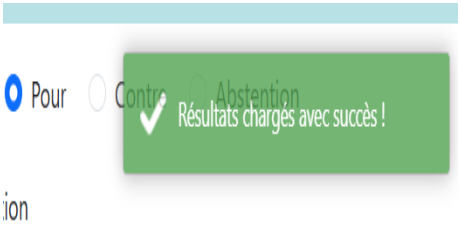


- **Formulaire QCM et options** :
- **Confirmation de soumission et résultats** :
- **Graphique des similarités** :

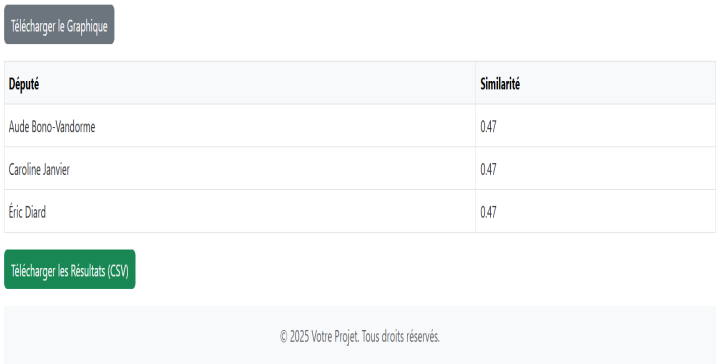
(a) Formulaire QCM



(b) Options disponibles

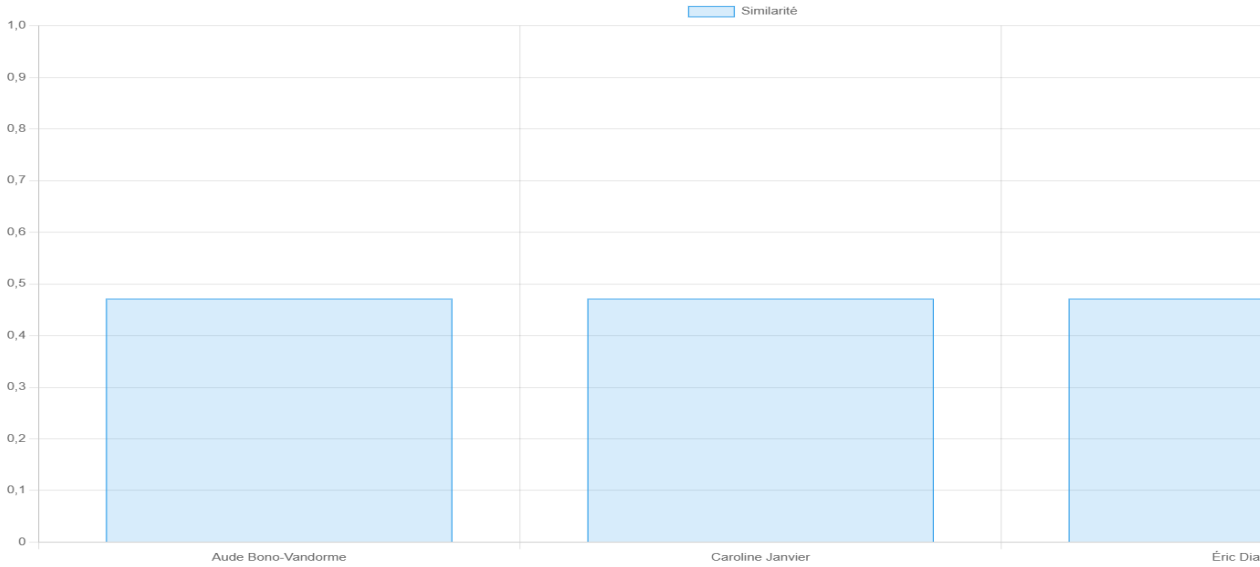


(a) Confirmation de soumission



Député	Similarité
Aude Bono-Vandorme	0.47
Caroline Janvier	0.47
Éric Diard	0.47

(b) Résultats des députés similaires



Télécharger le Graphique